

# VIBE CODING

VS

# CONTEXT ENGINEERING

VS

# SPEC-DRIVEN DEVELOPMENT

Ron Dagdag

Microsoft AI MVP / Research Engineering Manager at Thomson Reuters



**Microsoft®**  
Most Valuable  
Professional

# Ron Dagdag

Code > Create > Coach > Repeat.

Research Engineering Manager at

“Opinions expressed here are my own.  
I am not an agent of my employer”



**Thomson  
Reuters™**

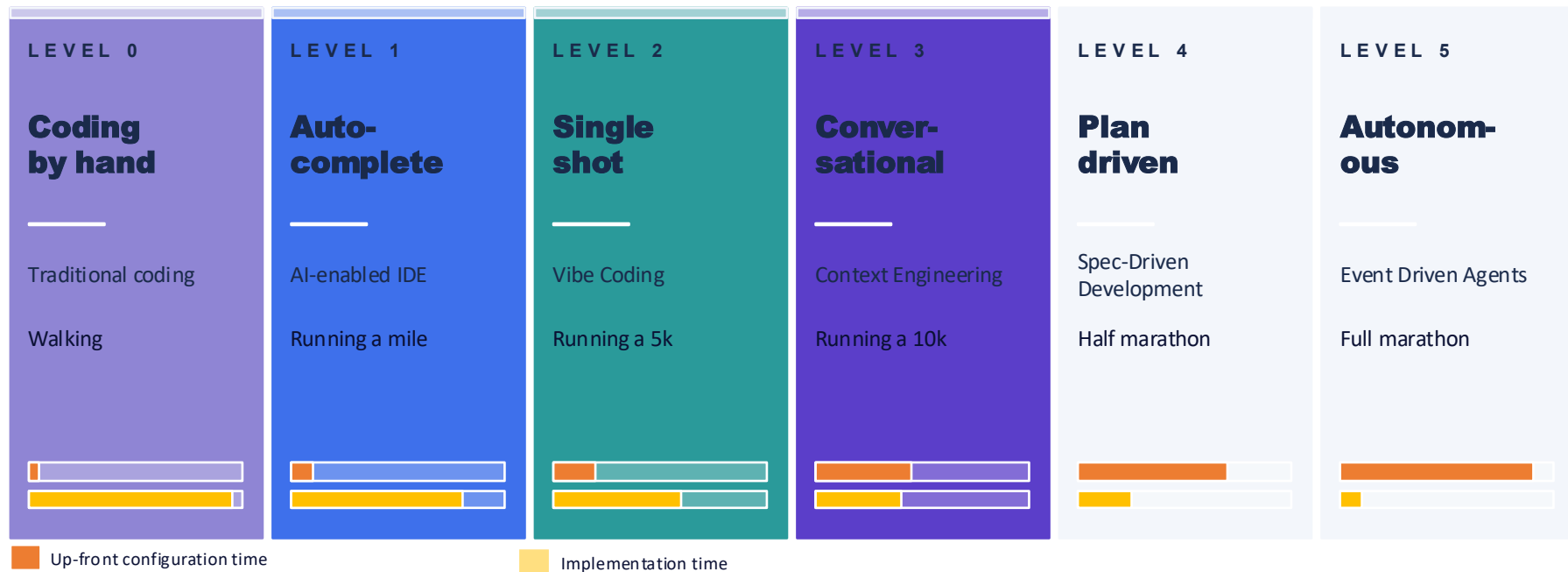


## Award Categories

AI, Windows Development,  
Internet of Things, Mixed Reality

# AI adoption hierarchy

*A framework for engineers to upskill — from manual coding to autonomous AI development*



# Four Approaches to AI-Assisted Development



## Vibe Coding

---

Fast, intuitive



## Context Engineering

---

Consistent, informed



## Spec-Driven Dev

---

Governed, reviewable



## Event-Driven Coding Agents

---

Async, like a teammate

Exploration



Production at Scale

# The GPS Analogy



## Vibe Coding

*"Take me somewhere nice"*

---

You might end up at a great  
restaurant.  
Or a parking lot.

## Context Engineering

*"Downtown, avoid  
highways,  
I like Italian"*

---

You'll get good options.  
Mostly right.

## Spec-Driven

*"123 Main St, arrive by  
5pm,  
via I-95, no tolls"*

Exact destination. Verified.  
Repeatable.

# Our Demo App: VoteJam

A party playlist voting app — same app, three ways to build it



**Submit songs**

Name + artist

**Vote up/down**

One vote per user per song

**Live rankings**

Songs sorted by votes

**Auth required**

Only logged-in users vote

React + Node/Express + PostgreSQL

# Our Journey Today

01

## Vibe Coding

What it is + live demo with Lovable

02

## Context Engineering

Skills, Knowledge Items & Artifacts in Google Antigravity

03

## Spec-Driven Dev

Contracts & CI pipelines with spec-kit + Claude Code

04

## Event-Driven Coding Agents

Copilot coding agent + @claude in GitHub

05

## The Playbook

When to use each + templates for your team

# 01

## Vibe Coding

Fast, intuitive, conversation-driven AI coding

---

# What Is Vibe Coding?

*"You describe what you want in natural language.  
The AI writes the code. You iterate by vibing."*

---

- **Conversational prompts — describe, don't specify**
- **Rapid iteration by feel — adjust on the fly**
- **Zero/minimal upfront planning — start from a blank slate**
- **AI has broad freedom to interpret your intent**



# Vibe Coding Tools

## Lovable

DEMO

Full-stack app generation from prompts. Deploys instantly.

## v0 by Vercel

UI component generation. React + Tailwind output.

## Bolt.new

Browser-based AI app builder with full environment.

## Replit Agent

End-to-end app creation in Replit's cloud IDE.

## Claude Artifacts

Interactive prototypes inside Claude conversations.

## ChatGPT Canvas

Code editing and iteration in OpenAI's interface.



# Live Demo: Vibe Coding with Lovable

Building VoteJam from a chat prompt

---

Speed | No tests | No conventions | No owner



Ron's Lovable



Home

Search



K

Resources

Connectors

Projects

All projects

Starred



Share Lovable

100 credits per paid referral



Upgrade to Pro

Unlock more features



Promo

Credits go 2x further until Apr 30 →

## What should we build, Ron?

Build me a party playlist voting app called "VoteJam":

- Fun, colorful UI with a music/party theme
- Text input to submit song name + artist
- List of songs ranked by votes
- Upvote/downvote buttons on each song
- Simple name entry for who submitted
- Confetti animation when a song gets 10+ votes

Make it fun with gradients and emoji.



Build ▾



My projects

Templates

Browse all →

# AIM Framework Breakdown

A structured prompt formula: Actor · Input · Mission



ACTOR

Persona

Define who the AI is acting as — the role, expertise level, and perspective it should adopt.

*e.g. "You are an expert marketing consultant" or "Act as a senior Python developer"*



INPUT

Context

Provide raw material, background data, constraints, or examples required to complete the task.

*e.g. Paste a document, dataset, code snippet, or describe the situation in detail*

M

MISSION

Goal

Clearly define what the AI should do with the input — the action, format, and output expected.

*e.g. "Draft a 500-word blog post," "Refactor this code," or "Summarize the key points"*

# Vibe Coding Tips

Move fast with AI · Best for prototypes, demos, and exploration

01

## Start with intent, not implementation

Describe what you want to build, not how. Let the AI propose structure.

03

## Use 'continue' sparingly

Restate context each prompt. AI forgets; you must be the memory.

05

## Accept 80% then polish

Get it roughly right fast, then manually fix the last 20% yourself.

02

## Iterate in small bursts

Prompt → preview → tweak. Never request more than one feature at a time.

04

## Screenshot the vibe

Paste UI references or describe the aesthetic — mood boards work better than specs.

06

## Commit before experimenting

Always git commit before a vibe session so you can roll back.

# Vibe Coding: The Verdict



## Use it for

- Spikes & prototypes
- Learning new APIs/tools
- Throwaway scripts
- Creative exploration



## Don't use it for

- “Production” features
- Security-sensitive code
- Long-lived codebases
- Anything that needs code review

# 02

## Context Engineering

Giving the AI the right information at the right time

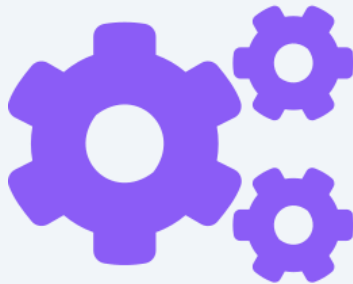
---

# What Is Context Engineering?

*"Designing the information environment around the AI so it produces the right output by default."*

---

- **Feed it your documentation & style guides**
- **Include API schemas and type definitions**
- **Set tool permissions and constraints**
- **Provide examples of correct output**



# The Iceberg of AI Output Quality

10% · What most developers optimize

**Your Prompt**

▼ 90% · What actually determines output quality

Context Files (CLAUDE.md · agents.md · .cursorrules)

Schemas & Types (OpenAPI · TypeScript interfaces · DB schemas)

Golden Examples (2-3 reference files: 'write code that looks like this')

Permissions & Constraints (what the AI can and can't touch)

Formal Spec (acceptance criteria · test scenarios · non-goals)

*Prompt engineering optimizes the 10%. Context engineering owns the 90%.*

# What the AI Actually Sees



## ✓ Inside the Context Window

Your prompt

Skills / agents.md / CLAUDE.md

Schema files you included

Code examples you provided

The file currently open

## ✗ Invisible to the AI

3 years of git history

Architecture docs on the wiki

Last week's design decisions

Your team's implicit conventions

That bug fix from Tuesday

*The AI can only reason about what you put in the window. Context engineering is choosing what goes in.*

# Optimize Your Context Window

Four dimensions that determine AI output quality



## Correctness

Is the context accurate?

Outdated docs = hallucinated  
code.

Wrong schemas = wrong output.



## Completeness

Does the AI have everything it  
needs?

Missing auth rules = missing  
auth.

Missing examples = inconsistent  
style.



## Size

Too much context = noise.

Too little = gaps.

Curate, don't dump.



## Trajectory

Does the context guide the AI  
toward your goal? Include not  
just  
WHAT but WHERE you're going.



# The New Hire Analogy

## Day 1 — no onboarding

"Here's the ticket. Go build it."

Invents their own naming conventions

Guesses which patterns to follow

Misses security rules nobody told them

Uses a different framework than the team

Code review: painful and slow

## Day 1 — full onboarding packet

Style guide, schemas & code examples

Follows established naming conventions

Applies team patterns automatically

Security rules explicitly stated

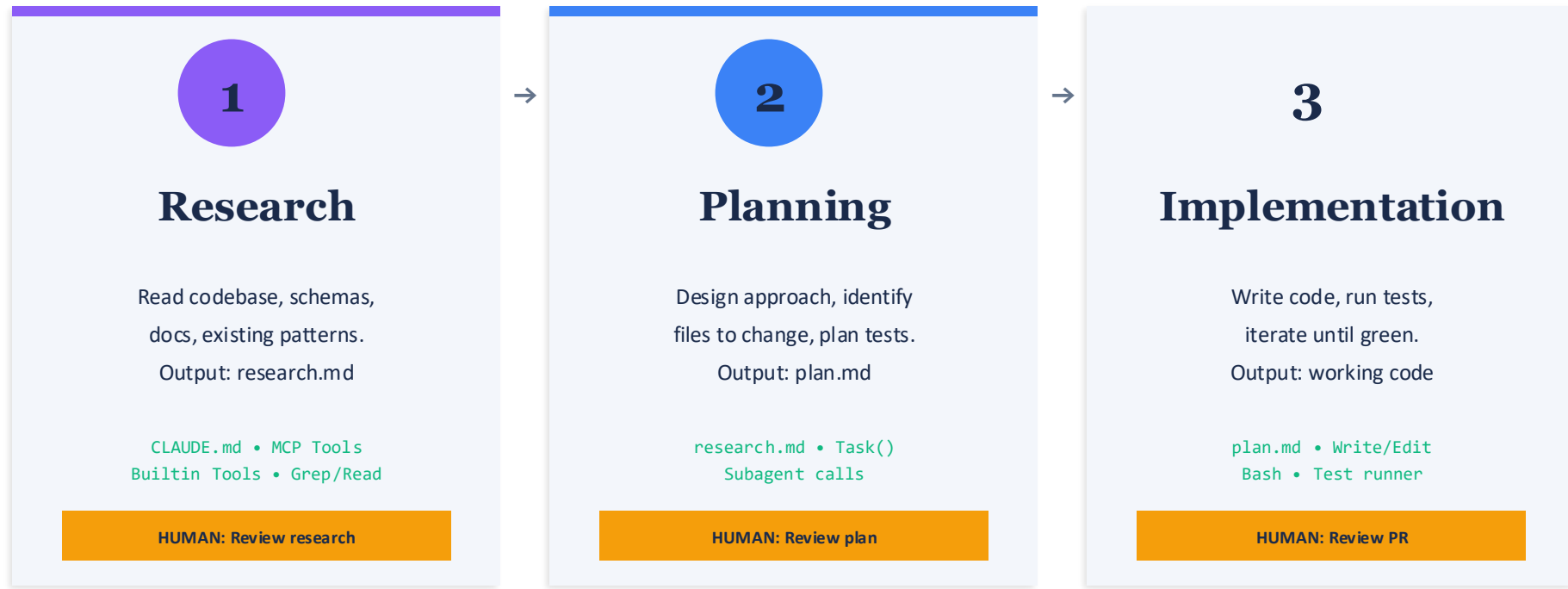
Auth middleware added without asking

Code review: spec check, not line-by-line

*Context engineering = the onboarding packet you give to the AI.*

# The Three-Phase Agent Workflow

*Good research → Good plans → Good code (bad research → bad everything)*



# The Four Pillars of Context



## Docs & Standards

Style guides,  
architecture docs,  
naming conventions



## Schemas & Types

API specs, TypeScript  
types, DB schemas



## Permissions

What the AI can and  
cannot touch



## Examples

Gold-standard code the  
AI should emulate

# Context Engineering Tools

## GitHub Copilot

DEMO

copilot-instructions.md + @workspace context in VS Code.

## Google Antigravity

Skills, Knowledge Items, Artifacts. Agent-first IDE.

## Claude Code

CLAUDE.md for project context. Terminal-based AI.

## Cursor

.cursorrules for project-level AI instructions.

## Windsurf

Cascade agent with codebase-wide context awareness.

## Aider

CLI tool. Git-aware. Repo map for context.



# Live Demo: Context Engineering

GitHub Copilot + copilot-instructions.md + @workspace

---

Consistency | Persistent memory | Transparent reasoning

**EXPLORER**

- OPEN EDITORS
- VOTEJAM
  - .github
    - skills
      - backend-api
        - SKILL.md
      - frontend-ui
    - copilot-instructions...
    - .playwright-mcp
    - dist
    - node\_modules
    - src
    - .env
    - .gitignore
    - package-lock.json
    - package.json
    - README.md
    - tsconfig.json
    - vitest.config.ts
  - OUTLINE
  - TIMELINE

**copilot-instructions.md**

```
1 # VoteJam - GitHub Copilot Instructions
2
3 > This file is automatically loaded by GitHub Copilot for
4 > every chat and inline suggestion
5 > in this repository. It is the team's AI onboarding packet -
6 > write it once, every session benefits.
7
8 ---
9
10 ## Project Overview
11
12 **VoteJam** is a party playlist voting API. Users submit
13 songs and vote them up or down.
14 Highest-voted songs surface first. Built for live demo
15 purposes; production-quality patterns apply.
16
17 ### Tech Stack
18 - **Runtime**: Node.js 20+, TypeScript (strict mode)
19 - **Framework**: Express.js
20 - **Validation**: Zod (all input, always)
21 - **Testing**: Vitest + Supertest
22 - **Auth**: Bearer token via `Authorization: Bearer <token>`
23 header
24 - **DB**: In-memory repository pattern (swap to PostgreSQL
25 without changing route logic)
26
27 ### Key Files
28 ```
29 src/
30   routes/songs.ts      ← Route handlers - follow this
31   pattern exactly
32   middleware/auth.ts    ← requireAuth middleware - attach
33   to all mutations
34 ```
```

**CLAUDE CODE** **CHAT**

**SESSIONS**

No matching sessions - [Reset Filter](#)

**Tip:** Use [/create-agent](#) to scaffold a custom agent for your workflow.

+ copilot-instructions.md

@workspace Create a new POST endpoint for voting on songs.  
POST /api/v1/songs/songId/vote with { direction: "up" | "down" }.

  - One vote per user per song
  - Server-side rate limiting
  - Include a test file

+ Agent Claude Sonnet 4.5

Local Default Approvals

Ln 8, Col 14 Spaces: 2 UTF-8 LF {} Instructions Go Live Prettier

# Context Engineering Tips

Shape what the AI knows · The craft of feeding the right context at the right time

01

## Window = Working Memory

The context window is the AI's only memory. Treat it like RAM — what's in it matters.

03

## Prune ruthlessly

Remove irrelevant history. Long context with noise beats short context without it — but clean wins.

05

## Show, don't just tell

Include 1–2 input/output examples inline. Few-shot beats zero-shot on complex tasks every time.

02

## Front-load critical facts

Put the most important constraints at the top of the prompt, not buried at the end.

04

## Use structured delimiters

Separate sections with `###` or XML tags. "Here is the code: `<code>...`" beats a wall of text.

06

## Summarise prior turns

In long sessions, compress earlier conversation into a one-paragraph brief and prepend it.

# 03

## Spec-Driven Development

Contracts, checklists, and CI-enforced pipelines

---

# What Is Spec-Driven Development?

*"Write the contract before writing the code.  
Let AI generate implementation from verifiable specs."*



- Specs define WHAT, AI generates HOW
- Every PR has a traceable spec origin
- CI checks enforce contract compliance
- Humans review specs, not every line of code

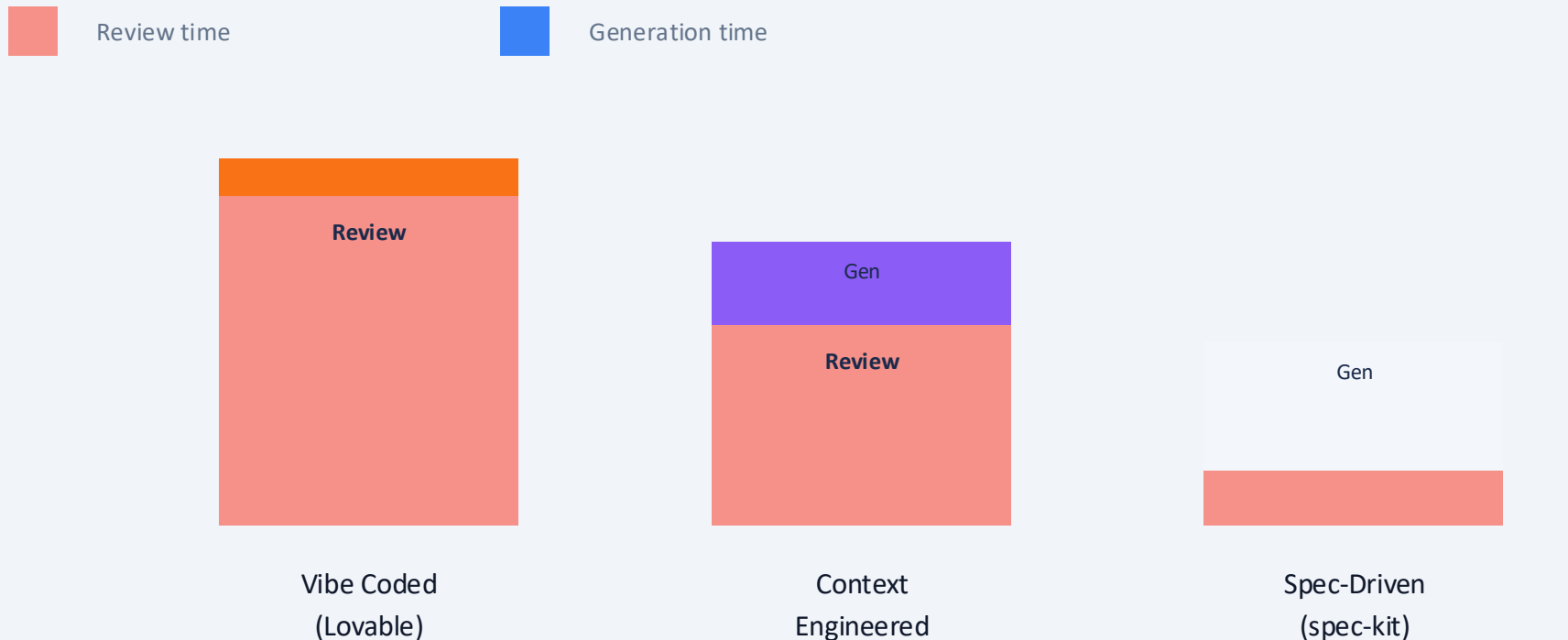
# Code Review: Hierarchy of Needs



Source: @blakesmith — "Code Review Essentials for Software Teams"



# The Hidden Bottleneck: Code Review



*AI made generation fast. Code review is now the bottleneck — and specs are the unlock.*

# The Division of Labor



## Humans Excel At

- Defining requirements
- Setting constraints & trade-offs
- Reviewing spec compliance
- Making architectural decisions



## AI Excels At

- Generating boilerplate & plumbing
- Implementing known patterns
- Writing test cases from specs
- Handling repetitive transformations



# Anatomy of a Good Spec

1

## Interface Contract

Input/output types, endpoint signatures, data shapes — the API the AI must satisfy

2

## Acceptance Criteria

Testable, specific conditions. Not 'it should work' — verifiable outcomes per scenario

3

## Constraints

Security rules, performance budgets, compliance requirements — the guardrails

4

## Non-Goals

Explicitly what you're NOT building. Prevents AI from scope-creeping into a 6-hour impl

5

## Test Scenarios

Happy path + edge cases. Write these in the spec — they become the actual test file

# Spec-Driven Tools

## spec-kit + Github Copilot

DEMO

Spec-to-code pipeline via slash commands in VS Code.

## Devin

Autonomous AI dev. Takes specs, generates validated PRs.

## Factory AI (Droids)

Enterprise AI agents following specs and org standards.

## BMAD

Research agent. GitHub issues to validated patches.

## OpenHands

Open-source AI dev platform. Spec-driven execution.

## Codegen

Codebase-aware agent with full repo context.

# The Spec-Driven Pipeline

1

**Write Spec**

Interfaces, criteria,  
constraints

>

2

**Gen Tasks**

AI decomposes into  
work items

>

3

**Implement**

Code from spec  
+ context

>

4

**CI Validates**

Tests, lint,  
security, types

>

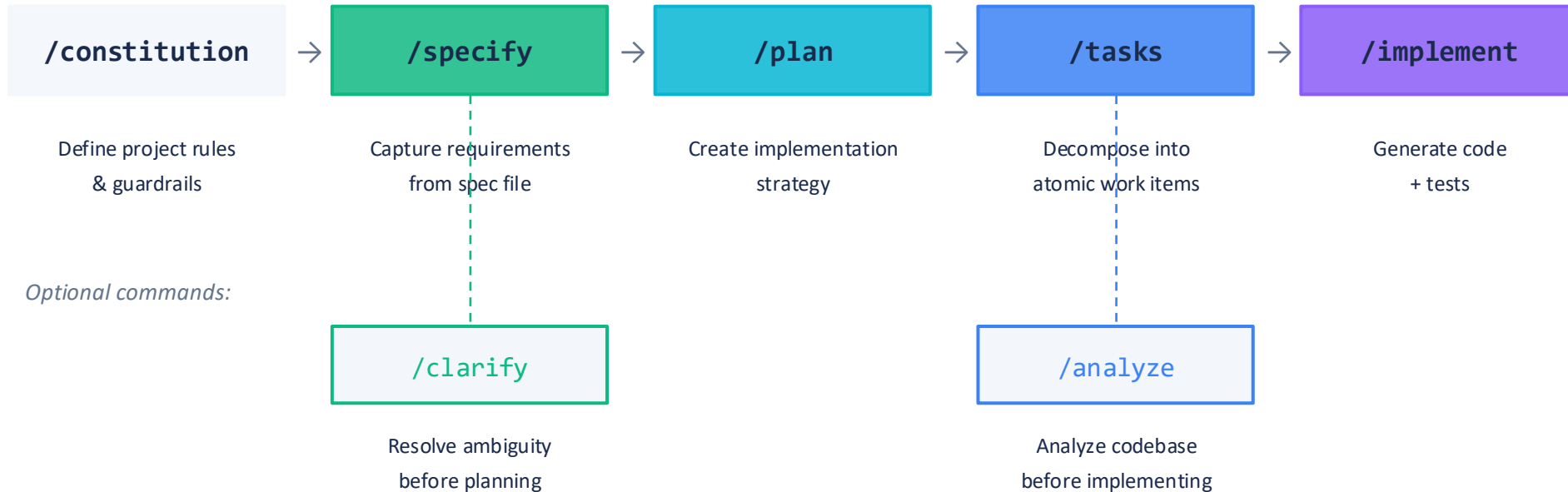
5

**Review**

Check spec  
compliance

# The spec-kit Commands

Run these slash commands inside Claude Code — in order.





# Live Demo: Spec-Driven with spec-kit

VS Code + Copilot — 60-line spec to 400+ lines of validated code

---

Contract | CI enforcement | Reviewable PRs

EXPLORER

> OPEN EDITORS

✓ VOTEJAM

- > .playwright-mcp
- > .specify
- > .vscode
- > dist
- > migrations
- > node\_modules
- ✓ specs/001-so...
- ✓ contracts
- ✓ spec.md
- ✓ src
- > \_\_tests\_\_
- > middleware
- > repositories
- ✓ routes
- > \_\_tests\_\_
- TS songs.ts 4
- > types
- > utils
- TS app.ts
- .env
- .gitignore
- CLAUDE.md
- ( ) package-lock.json
- ( ) package.json
- ① README.md
- ts tsconfig.json
- TS vitest.config.ts

> OUTLINE

> TIMELINE

< 4 0 Git Graph

CLAUDE.md Preview README.md X README.md

## What's Here

- **Types:** Song, ApiResponse
- **Middleware:** requireAuth (Bearer token auth), validateBody (Zod schema validation)
- **Repository:** In-memory song store with vote tracking
- **Routes:**
  - POST /api/v1/songs — Submit a new song (requires auth)
  - GET /api/v1/songs — List all songs (sorted by votes)
- **Tests:** 14 passing tests (vitest + supertest)

## Project Structure

```
src/
├── app.ts # Express app setup
```

PROBLEMS 4 TERMINAL PLAYWRIGHT PORTS AZURE ...

TERMINAL

```
Ron@MacBook-Pro-7 VoteJam % uvx --from git=https://github.com/
git specify init .
| report (after /speckit.tasks, before /speckit.implement)
|
| ◦ /speckit.checklist (optional) - Generate quality checklists to
|
| validate requirements completeness, clarity, and consistency (afte
| /speckit.plan)
|
|
```

Ron@MacBook-Pro-7 VoteJam %

CLAUDE CODE CHAT

SESSIONS

No matching sessions - [Reset Filter](#)

Tip: Use [/create-agent](#) to scaffold a custom agent for your workflow.

+ ① README.md

Describe what to build

+ ② Agent Claude Sonnet 4.6 - High

Local Default Approvals

Go Live

# Spec-Driven Dev Techniques

Spec first, then code · Like Red-Green-Refactor, but with AI as the implementer



## RED

Write the spec (tests / acceptance criteria) before any code exists. Expect failure — that's the point.



## GREEN

Ask AI to write minimal code that satisfies the spec. Just enough to pass.



## REFACTOR

Review AI output with your spec as the quality bar. Improve without breaking.

01

### Write specs in plain English

Define inputs, outputs, and edge cases before touching the AI. Use Given/When/Then format.

02

### Spec as system prompt

Paste the full spec into the AI system prompt. The AI writes against it, not your vague intent.

03

### Failing test = failing prompt

Run tests after each AI output. Feed failures back to the AI with the failing case as context.

04

### Freeze spec, not code

Specs are sacred; code is disposable. Regenerate any time — never change the spec to fit bad code.

# Spec-Driven: The Verdict



## Use it for

- Production features with compliance needs
- Long-lived, team-maintained code
- Anything requiring a real security audit
- Features where review cost matters most



## Skip the formal spec for

- Quick experiments & throwaway scripts
- When there's no prior type system or schema
- Solo/no-review projects
- Early-stage exploration work

# 04

## Event-Driven Coding Agents

AI that works on issues and PRs while you sleep

---



# The Night Shift Teammate

## Today: You're in the loop

You write a prompt

AI responds

You read & evaluate

You re-prompt or edit

You paste code in editor

Repeat for every task

Blocked when you're offline

## Event-Driven: You review

You write a clear GitHub issue

You close your laptop

Agent researches codebase

Agent plans & implements

Agent opens a draft PR

You come back to a review

Works while you sleep

*The shift: from CODER who uses AI → MANAGER who reviews AI output.*

# Event-Driven Coding Agents

*"Assign an issue. Wake up to a PR."*



- AI agents work asynchronously — like a teammate, not a tool
- They research your codebase, plan an approach, implement, and open a PR
- You review and merge — the agent handles the implementation
- Two flavors today: GitHub Copilot coding agent + @claude in GitHub



# What 'Event-Driven' Actually Means

*The agent sleeps until an event wakes it up — you never open a chat window*

EVENT

## Issue Assigned

Assign to Copilot  
in the issue sidebar

### Copilot Coding Agent

Reads codebase →  
Plans approach →  
Opens Draft PR

EVENT

## @claude/@copilot mention

Type @claude in any  
issue or PR comment

### @claude/@copilot GitHub Action

Reads thread →  
Investigates code →  
Responds or commits

EVENT

## PR Comment

'@copilot fix this  
endpoint validation'

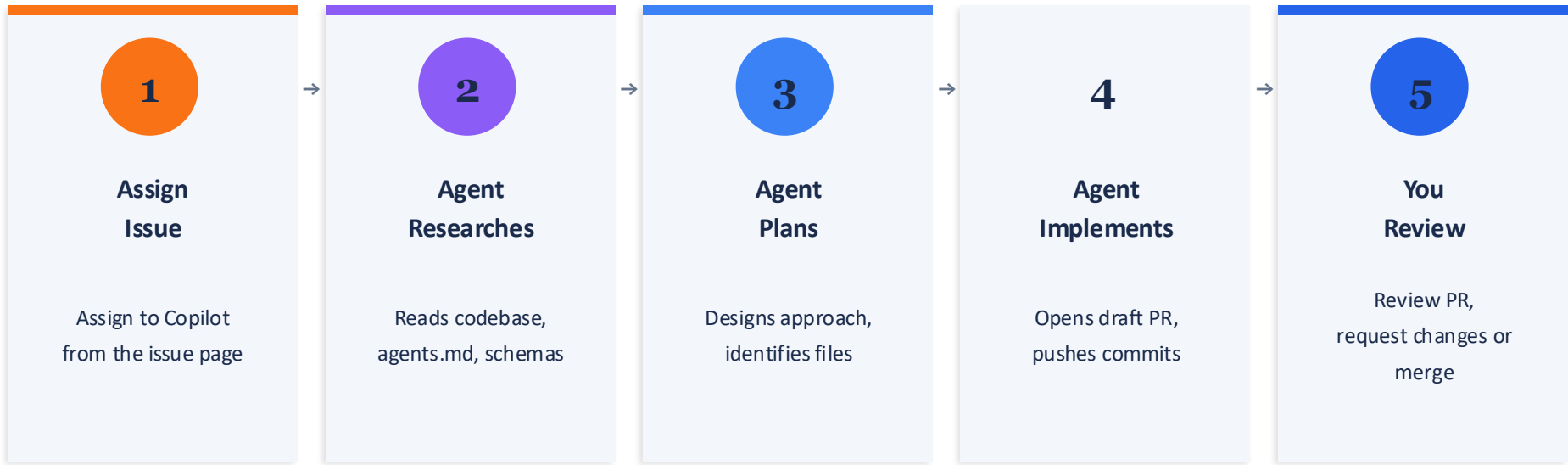
### @claude/@copilot GitHub Action

Reads comment →  
Pushes fix commit →  
Updates PR branch

*'Event-driven' = triggered by work items in your existing workflow, not by opening a chat window.*

# GitHub Copilot Coding Agent

*Built into GitHub — assign an issue, get a PR*



Setup: `copilot-setup-steps.yml` workflow + `agents.md` + Copilot Enterprise/Business

The agent runs in a GitHub-hosted environment. Your context files (`agents.md`, `schemas`) travel with the repo.

# Write Issues Agents Can Execute

*Your issue IS the spec for the agent — quality in, quality out*

## ✗ Vague Issue (agent guesses)

Fix the search.

The search doesn't  
work properly.

Please fix it.

## ✓ Agent-Ready Issue

Add song search to VoteJam

User can search by title or artist.

Criteria:

- Case-insensitive match
- Instant results (no submit)
- Empty query = all songs

Files: routes/songs.ts

Tests required.

*Think of the issue as a spec: user story + acceptance criteria + file hints = agent that delivers.*



# Live Demo: Copilot Coding Agent

Assign an issue → Agent opens a PR

---

Agents Tab | Async | Issue → PR | Auto-review



# 05

## The Playbook

When to use each approach + templates for your team

---

# When to Use What

|                | Vibe Coding | Context Eng | Spec-Driven | Coding Agents     |
|----------------|-------------|-------------|-------------|-------------------|
| Speed          | Fastest     | Fast        | Moderate    | Async (hours)     |
| Consistency    | Low         | Medium-High | High        | High              |
| Governance     | None        | Informal    | Formal      | Automated         |
| Risk Tolerance | High        | Medium      | Low         | Low               |
| Best For       | Exploration | Feature dev | Production  | Issue backlogs    |
| Review Cost    | High        | Medium      | Low (spec)  | Agent-created PRs |
| Key Artifact   | Prompt      | agents.md   | Spec file   | GitHub Issue      |

Match the approach to the risk profile of the task.



# But... (Addressing the Pushback)

***"Writing specs is too slow."***

→ A spec takes 20 min. Review of unspecced code takes 4 hours. Do the math.

***"The AI should figure this out itself."***

→ The AI is a spotlight, not a mind reader. Design the context; don't hope for magic.

***"This is over-engineering."***

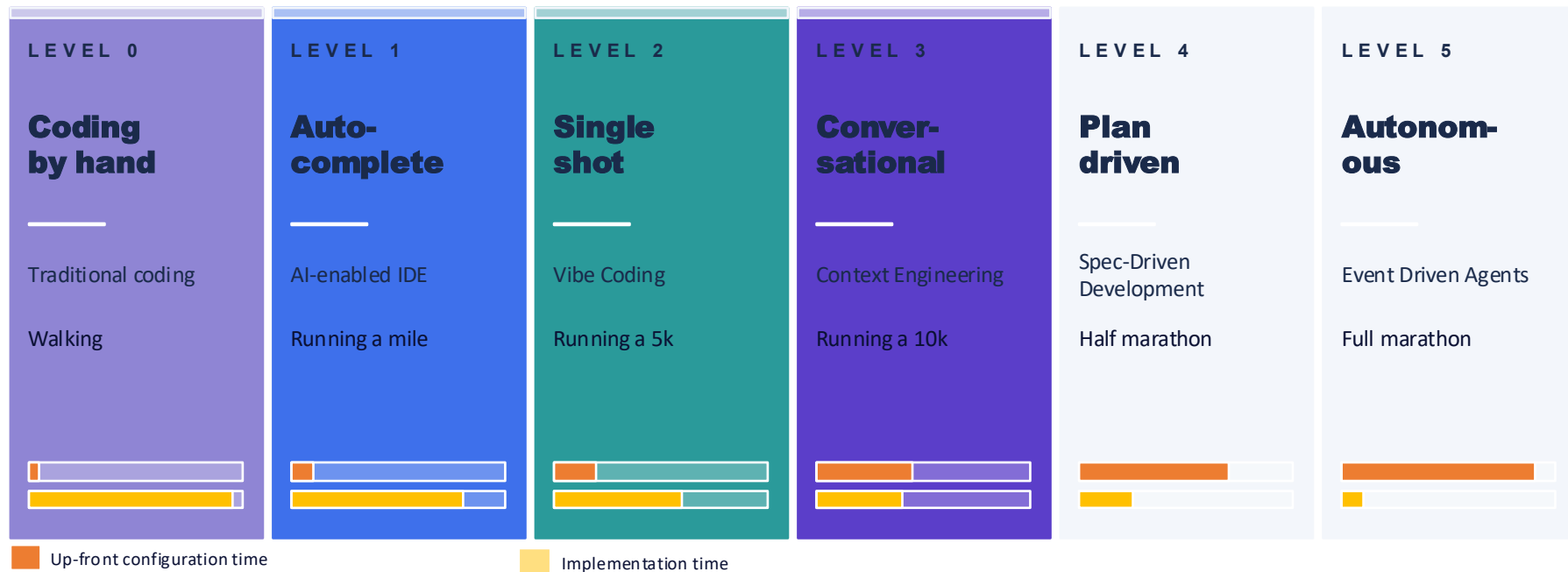
→ Vibe code prototypes, spec production code. Match the formality to the stakes.

***"Our team won't adopt this."***

→ Start with one agents.md file. Show the output difference. Let the code sell it.

# AI adoption hierarchy

*A framework for engineers to upskill — from manual coding to autonomous AI development*



# Four Takeaways

1

## Match the method to the risk

✓ **Use when:** prototyping, exploring ideas, low-stakes experiments. **Write a red test first** → AI turns it green.  
✗ **Avoid:** shipping to production or working in a shared codebase.

2

## Context is the multiplier

✓ **Use when:** iterating on real features; pair with any method. **Include tests in context** → AI verifies its own output.  
✗ **Avoid:** context is vague — garbage in, garbage out still applies.

3

## Let agents work while you sleep

✓ **Use when:** spec is well-defined; tasks are decomposable and testable. **Specs become test suites** → agent loops until green.  
✗ **Avoid:** requirements are fuzzy — agents amplify ambiguity, not resolve it.

4

## AIM + Red/Green TDD apply to all three

**AIM:** Actor, Input & Mission — the prompt framework that works across every method.  
**TDD:** Write the failing test first; AI verifies itself by making it pass. Works across all three.

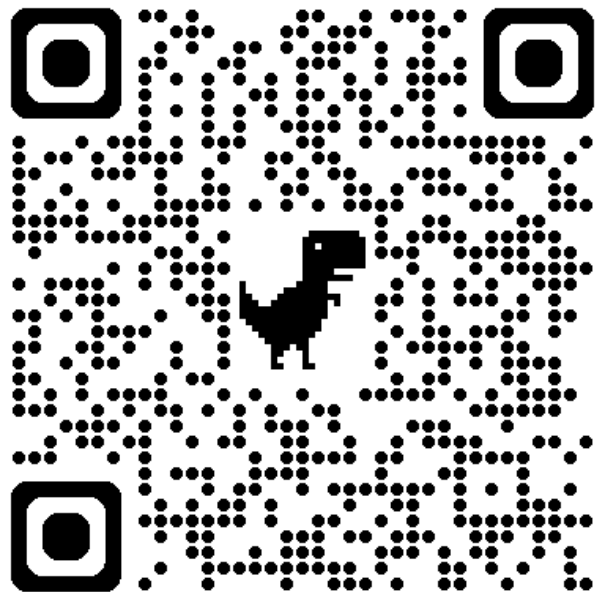
**VIBE CODING**

**VS**

**CONTEXT  
ENGINEERING**

**VS**

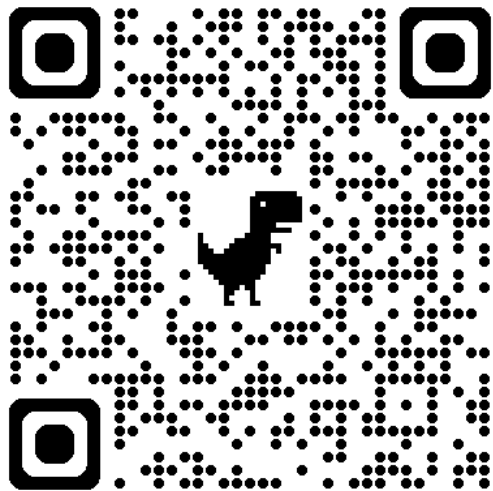
**SPEC-DRIVEN  
DEVELOPMENT**



# Ron Dagdag

Code > Create > Coach > Repeat.

<https://www.linkedin.com/in/rondagdag/>



Research Engineering Manager at



**Thomson  
Reuters™**



**Microsoft®**  
Most Valuable  
Professional



**Award Categories**

AI, Windows Development,  
Internet of Things, Mixed Reality